

Guia para comprender la simulacion

Polideportivo Alberdi - logica de resolucion y lectura fila por fila

Objetivo: entender primero como piensa el motor, como se resuelve cada evento y como se construye una fila del vector de estado. La parte visual de Streamlit se presenta al final como una capa separada.

1. La idea central

El programa implementa una simulacion de eventos discretos. El reloj no avanza minuto a minuto. En cada iteracion se busca el evento futuro con menor hora, el reloj salta directamente hasta esa hora, se actualizan acumuladores, se procesa el evento y se registra una nueva fila.

La regla mental para leer el vector de estado es:

Fila anterior -> elegir el menor evento futuro -> avanzar el reloj -> ejecutar todas las consecuencias -> escribir la nueva fila.

Importante: la fila muestra el estado despues de procesar el evento que figura en su columna Evento.

2. Eventos administrados

- Llegada Futbol.
- Llegada HandBall.
- Llegada Basket.
- Fin Uso Cancha.
- Fin Limpieza.
- Fin Simulacion.

Los eventos que no estan programados tienen valor infinito. Por ejemplo, si nadie usa la cancha, Fin Uso Cancha queda en infinito. La funcion `_evento_minimo()` compara el diccionario de eventos y selecciona la hora menor.

3. Estado del sistema

El diccionario estado funciona como la memoria completa de la simulacion. Contiene:

- Reloj actual.
- Estado de la cancha: Libre, Ocupada o En Limpieza.
- Disciplina y grupo que estan usando la cancha.
- Una cola por disciplina.
- Objetos temporales correspondientes a los grupos activos.
- Horas de todos los eventos futuros.
- Acumuladores de espera, tiempo libre y tiempo ocupado.

- Contadores de atendidos, retirados y limpiezas.

4. Distribuciones utilizadas

Variable	Distribucion	Formula predeterminada
Llegada Futbol	Exponencial negativa	$X = -600 \ln(1 - \text{RND})$
Llegada HandBall	Uniforme	$X = 600 + \text{RND}(840 - 600)$
Llegada Basket	Uniforme	$X = 360 + \text{RND}(600 - 360)$
Uso Futbol	Uniforme	$X = 80 + \text{RND}(100 - 80)$
Uso HandBall	Uniforme	$X = 60 + \text{RND}(100 - 60)$
Uso Basket	Uniforme	$X = 70 + \text{RND}(130 - 70)$

Un tiempo entre llegadas es una duracion. Para programar la llegada absoluta se suma al reloj: proxima llegada = reloj actual + tiempo entre llegadas.

5. Inicializacion con semilla 42

Para seguir numeros reproducibles se ejecuto el motor con semilla 42 y parametros predeterminados. En el instante cero se generan las tres primeras llegadas.

Disciplina	RND	Calculo	Hora programada
Futbol	0.7740	$-600 \ln(1 - 0.7740)$	892.2155
HandBall	0.4389	$600 + 0.4389 \times 240$	705.3308
Basket	0.8586	$360 + 0.8586 \times 240$	566.0635

La fila de inicializacion queda conceptualmente asi:

```
N = 0
Reloj = 0
Evento = Inicializacion
Cancha = Libre
Proxima Basket = 566.0635
Proxima HandBall = 705.3308
Proxima Futbol = 892.2155
```

Como 566.0635 es la menor de las horas programadas, el primer evento real sera Llegada Basket.

6. Ejecucion completa de la fila N = 1

Evento: Llegada Basket. Hora del evento: 566.0635.

Paso A - Avanzar el reloj

El reloj pasa de 0 a 566.0635. Como la cancha estuvo Libre durante todo el intervalo, ese delta se suma al tiempo libre acumulado.

```
delta = 566.0635 - 0 = 566.0635
tiempo libre acumulado = 566.0635
```

Paso B - Programar la siguiente llegada de Basket

Cada llegada programa inmediatamente la siguiente llegada de su misma disciplina. El nuevo RND es 0.6974.

```

tiempo entre llegadas = 360 + 0.6974 x 240 = 527.3683
proxima llegada Basket = 566.0635 + 527.3683 = 1093.4318

```

Paso C - Verificar capacidad y crear el grupo

La cola total vale 0 y la capacidad maxima es 5. Como 0 no es mayor ni igual que 5, el grupo no se retira. Se crea G1, disciplina Basket, inicialmente En Cola, con hora de llegada 566.0635.

Paso D - Iniciar el uso de la cancha

Como la cancha estaba Libre, G1 es seleccionado inmediatamente. Sale de la cola, cambia a En Cancha y su espera queda en cero.

```

tiempo espera G1 = 566.0635 - 566.0635 = 0
RND uso = 0.0942
tiempo uso Basket = 70 + 0.0942 x 60 = 75.6506
fin de uso = 566.0635 + 75.6506 = 641.7141

```

Resultado de la fila N = 1

Campo	Valor despues del evento
Reloj	566.0635
Evento	Llegada Basket
Estado cancha	Ocupada
Disciplina / grupo actual	Basket / G1
Cola Basket	0
Proxima llegada Basket	1093.4318
Proximo fin de uso	641.7141
Tiempo libre acumulado	566.0635
Objeto activo	G1-Basket-EnCancha

La cola Basket aparece en cero porque G1 se agrega a la cola y, en la misma ejecucion del evento, se retira para entrar a una cancha que estaba libre.

7. Ejecucion de la fila N = 2

Evento: Fin Uso Cancha. Hora del evento: 641.7141.

El reloj avanza desde 566.0635 hasta 641.7141. El delta es 75.6506 y, como la cancha estaba Ocupada, se suma al tiempo ocupado acumulado.

```

delta = 641.7141 - 566.0635 = 75.6506
tiempo ocupado acumulado = 75.6506

```

Luego el grupo G1 finaliza:

- Su hora de salida se fija en 641.7141.
- Su espera, que era 0, se agrega al acumulador de Basket.
- El contador de atendidos Basket pasa a 1.
- G1 se elimina de los objetos temporales activos.
- Se calcula el tiempo de limpieza y la cancha pasa a En Limpieza.

8. Resolucion numerica de la limpieza

La ecuacion diferencial usada es:

$$dD/dt = 0.6 C + t, \text{ con } D(0) = 0$$

C es la cantidad total de grupos que esperan al comenzar la limpieza. Para la primera limpieza $C = 0$, el objetivo de Basket es $D = 300$ y $h = 0.1$. Entonces la derivada queda $dD/dt = t$.

Metodo de Euler

$$D \text{ siguiente} = D \text{ actual} + h (0.6 C + t \text{ actual})$$

Paso	t actual	D actual	f(t,D)	Incremento	D siguiente
1	0.0	0.00	0.0	0.00	0.00
2	0.1	0.00	0.1	0.01	0.01
3	0.2	0.01	0.2	0.02	0.03
4	0.3	0.03	0.3	0.03	0.06

El ciclo continua hasta que D alcanza o supera 300. Se requieren 246 pasos: $t = 24.6$ y $D \text{ final} = 301.35$. Por eso el fin de limpieza se programa en $641.7141 + 24.6 = 666.3141$.

Metodo RK4

Si se selecciona RK4, cada paso calcula cuatro pendientes intermedias k_1, k_2, k_3 y k_4 . Luego combina esas pendientes para estimar mejor el incremento. El motor elige entre Euler y RK4 mediante el parametro `metodo_integracion`.

9. Fin de limpieza

Cuando ocurre Fin Limpieza, la cancha pasa a Libre, el evento Fin Limpieza vuelve a infinito y el motor intenta seleccionar el siguiente grupo. Si hay alguien esperando, inicia su uso en esa misma fila; si no, la cancha permanece libre.

10. Regla de prioridad

La funcion `seleccionar_proximo_grupo` implementa una prioridad especifica:

- Si esperan grupos de Futbol o Basket, se comparan los primeros de esas dos colas y gana el que llego antes.
- HandBall se selecciona solamente cuando no espera ningun grupo de Futbol ni de Basket.
- Dentro de cada disciplina se mantiene el orden FIFO: primero en llegar, primero en salir.

11. Objetos temporales

Cada grupo es un objeto temporal mientras esta esperando o usando la cancha. Guarda id, disciplina, estado, hora de llegada, inicio de uso, salida y espera. Ejemplos de resumen:

```
G3-Futbol-EnCancha
G4-Basket-EnCola
```

Al finalizar el uso, el objeto se elimina. Sus datos relevantes ya quedaron incorporados en acumuladores y contadores.

12. Acumuladores, contadores y metricas

Resultado	Formula
Espera promedio por disciplina	Acumulador de espera / cantidad atendida
Dias simulados	Tiempo simulado / 1440
Tiempo libre diario promedio	Tiempo libre acumulado / dias simulados
Limpiezas promedio por dia	Cantidad de limpiezas / dias simulados

El tiempo se acumula segun el estado previo de la cancha durante cada salto del reloj. Si estaba Libre suma tiempo libre; si estaba Ocupada suma tiempo ocupado. El tiempo En Limpieza no se incluye en ninguno de esos dos acumuladores.

13. Flujo completo del motor

- Crear el generador aleatorio y el estado inicial.
- Programar Fin Simulacion.
- Generar la primera llegada de cada disciplina.
- Guardar la fila $N = 0$ de Inicializacion.
- Buscar el evento con hora minima.
- Actualizar tiempos acumulados usando el delta del reloj.
- Procesar la llegada, el fin de uso o el fin de limpieza.
- Crear la fila con el estado resultante.
- Repetir hasta Fin Simulacion o hasta superar el maximo de iteraciones.
- Calcular las metricas finales y devolver DataFrames para la interfaz.

14. Mapa de archivos

Archivo	Responsabilidad
simulacion/motor.py	Ciclo de eventos, llegadas, usos, limpiezas y creacion de filas.
simulacion/estado.py	Estado inicial, objetos temporales, colas y prioridad.
simulacion/distribuciones.py	Transformacion de RND en tiempos de llegada y uso.
simulacion/integracion.py	Calculo del tiempo de limpieza mediante Euler o RK4.
simulacion/metricas.py	Calculo de promedios y resultados finales.
app.py	Parametros de Streamlit y presentacion visual de resultados.

15. Como interpretar cualquier fila

Frente a una fila del vector de estado, conviene responder en este orden:

- Que evento ocurrio y a que hora.
- Desde que hora venia el reloj y cual fue el delta.
- En que estado estuvo la cancha durante ese delta.
- Que acumulador aumento.
- Que objeto se creo, cambio o elimino.
- Que RND se uso y que variable produjo.
- Que nuevo evento futuro se programo.
- Como quedaron las colas, la cancha y los contadores despues del evento.

Si se sigue ese orden, cada fila deja de ser una tabla enorme y pasa a ser una historia corta: ocurrio un evento, cambio una parte del estado y quedaron programadas las proximas acciones.

16. Capa visual de la aplicacion

app.py no decide la logica del sistema. Su funcion principal es leer parametros desde la barra lateral, llamar a simular(parametros), reorganizar columnas para mostrarlas con encabezados agrupados y presentar metricas, vector de estado y tablas de integracion. Por eso, para comprobar la resolucion de los ejercicios, primero se debe mirar simulacion/motor.py y sus modulos auxiliares; app.py se estudia despues como capa de presentacion.